Datensäuberung & Feature Engineering (Pre-Processing)

Matrikelnummer: 2130238

Das Pre-Processing wird zentral in diesem Notebook durchgeführt, damit alle nachfolgenden Assignments mit demselben bereinigten Datensatz arbeiten. Das Notebook deckt dabei zwei Bereiche ab:

- **Datensäuberung:** Fehlende Werte, Ausreißer, inkonsistente Einträge
- **Feature Engineering:** Encoding kategorischer Variablen, Anlage der Jahreszeiten-Variable, Rundung

Der gesäuberte Datensatz wird am Ende als `dataset_cleaned.csv` exportiert.

Importieren von Bibliotheken

In [1]:
```
# Reguläre Python Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skew, kurtosis
```

Einlesen der Daten

In [2]:
```
df = pd.read_csv("../data/dataset.csv")
print("Dataset loaded successfully!")
print(f"Shape: {df.shape}")

# Speichern für die spätere Verwendung (Ende des Skripts)
rows_at_begin = df.shape[0]
```

```
Dataset loaded successfully!
Shape: (8662, 22)
```

Allgemeine Information der Daten und Säubern der Daten

In [3]:
```
df.head()
```

Out[3]:

	Customer_ID	CCreditScore	CGeography	CGender	CAge	CTenure	CBalance	C
0	CUST_000001	553.295220	France	Male	39	2	131008.168572	
1	CUST_000002	552.939394	France	Male	33	4	118529.788812	
2	CUST_000003	688.745934	Spain	Male	40	1	-879.724555	
3	CUST_000004	771.940664	France	Male	40	9	125563.132706	
4	CUST_000005	484.092560	France	Male	55	5	95759.918567	

5 rows × 22 columns

In [4]:

```
# Informationen der Spalten anzeigen: Null-Values und Datentypen
print(df.info())

# Nullwerte explizit ausgeben (absolut und prozentual)
null_counts = df.isnull().sum()
null_percent = (null_counts / len(df) * 100).round(2)
null_summary = pd.DataFrame({'Absolut': null_counts, 'Prozent (%)': null_percent
null_summary = null_summary[null_summary['Absolut'] > 0]

print("="*25, "Fehlende Werte pro Spalte", "="*25)
print(null_summary)
print("\nHinweis (Checklist 4.b.ii):")
print("  < 5-10%  → Imputation empfohlen")
print("  > 10-20% → Entfernen von Zeilen/Spalten ggf. notwendig")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8662 entries, 0 to 8661
Data columns (total 22 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer_ID               8662 non-null   object
 1   CCreditScore              8662 non-null   float64
 2   CGeography                8662 non-null   object
 3   CGender                   8662 non-null   object
 4   CAge                      8662 non-null   int64
 5   CTenure                   8662 non-null   int64
 6   CBalance                  8585 non-null   float64
 7   CNumOfProducts            8662 non-null   int64
 8   CHasCrCard                8662 non-null   int64
 9   CIsActiveMember           8662 non-null   int64
 10  CEstimatedSalary          7884 non-null   float64
 11  Days_Since_Onboarding     8662 non-null   float64
 12  Account_Age_Months        8662 non-null   float64
 13  Avg_Monthly_Transactions  8662 non-null   int64
 14  Transaction_Variance      8662 non-null   float64
 15  Last_Login_Days_Ago       8662 non-null   float64
 16  Support_Tickets_Count     8662 non-null   int64
 17  Mobile_App_Usage_Hours    8662 non-null   float64
 18  Onboarding_Month          8662 non-null   int64
 19  Is_Holiday_Onboarding     8662 non-null   int64
 20  Churn                     8662 non-null   int64
 21  CLV_Continuous            8662 non-null   float64
dtypes: float64(9), int64(10), object(3)
memory usage: 1.5+ MB
None
========================= Fehlende Werte pro Spalte =========================
                  Absolut  Prozent (%)
CBalance               77         0.89
CEstimatedSalary      778         8.98

Hinweis (Checklist 4.b.ii):
  < 5-10%  → Imputation empfohlen
  > 10-20% → Entfernen von Zeilen/Spalten ggf. notwendig
```

In [5]:

```python
# Duplikate finden und behandeln
dup_count = df[df.duplicated()].shape[0]
print("Es gibt", dup_count, "duplizierte Zeilen im DataFrame.")

if dup_count > 0:
    df.drop_duplicates(inplace=True)
    print(f"{dup_count} Duplikate wurden entfernt. Neue Anzahl Zeilen: {df.shape
else:
    print("Keine Duplikate vorhanden — keine Aktion notwendig.")
```

```
Es gibt 0 duplizierte Zeilen im DataFrame.
Keine Duplikate vorhanden — keine Aktion notwendig.
```

Überprüfe und Säubere Identifier

In [6]:

```python
# Prüfe Customer_ID Format und dass jeder Customer eine ID hat
print("Es gibt folgende fehlerhafte Customer_IDs, die nicht \"CUST\" im Namen be
print("Es gibt folgende fehlerhafte Customer_IDs, die nicht die länge von 6 Ziff
print("Es gibt folgende fehlerhafte Customer_IDs, die keine numerischen Werte au
```

```
Es gibt folgende fehlerhafte Customer_IDs, die nicht "CUST" im Namen besitzen:  0
Es gibt folgende fehlerhafte Customer_IDs, die nicht die länge von 6 Ziffern aufw
eisen:  0
Es gibt folgende fehlerhafte Customer_IDs, die keine numerischen Werte aufweisen:
0
```

Überprüfe und Säubere Customer Demographics

In [7]:
```
# Spaltennamen mit den erwarteten Spaltennamen (France, Germany, Spain) vergleic
print("Unique countries: ", df['CGeography'].unique())
```

```
Unique countries:  ['France' 'Spain' 'Germany' 'SPAIN' 'Fr' 'GERMANY' 'DE' 'spai
n' 'ES'
 'germany' 'france']
```

In [8]:
```
# Ordne jedem Land die korrekten Bezeichnungen zu
df['CGeography'] = df['CGeography'].replace({'Fr': 'France', "france": 'France'}
df['CGeography'] = df['CGeography'].replace({'GERMANY': 'Germany', "DE": 'German
df['CGeography'] = df['CGeography'].replace({'SPAIN': 'Spain', "spain": 'Spain',

# Überprüfe die einzigartigen Ländernamen erneut
print("Einzigartige Länder nach der Korrektur: ", df['CGeography'].unique(), "\n

# Wandle die Länder in categorical
df['CGeography'] = df['CGeography'].astype('category')
print("Der Datentyp von CGeography ist nun", df['CGeography'].dtype)
```

```
Einzigartige Länder nach der Korrektur:  ['France' 'Spain' 'Germany']

Der Datentyp von CGeography ist nun category
```

In [9]:
```
# Überprüfe visuell, ob das Gehalt mit dem Kredit-Score zusammenhängt pro Land
germany_data = df[df['CGeography'] == 'Germany'][['CCreditScore', 'CEstimatedSal
spain_data = df[df['CGeography'] == 'Spain'][['CCreditScore', 'CEstimatedSalary'
france_data = df[df['CGeography'] == 'France'][['CCreditScore', 'CEstimatedSalar

print("\nFür Deutschland sortiert nach CCreditScore (aufsteigend):")
germany_sorted = germany_data.sort_values('CCreditScore')
print(germany_sorted.head(10))

print("\nFür Spanien sortiert nach CCreditScore (aufsteigend):")
spain_data = spain_data.sort_values('CCreditScore')
print(spain_data.head(10))

print("\nFür Frankreich sortiert nach CCreditScore (aufsteigend):")
france_data = france_data.sort_values('CCreditScore')
print(france_data.head(10))
```

```
Für Deutschland sortiert nach CCreditScore (aufsteigend):
      CCreditScore  CEstimatedSalary
2108      0.000000      76204.573000
7275      0.000000     119122.582218
7082      0.000000     102184.456050
4371      0.000000       5659.346530
8576      0.000000     185977.261285
8170    402.120183     176625.357361
7080    402.376014      71966.790507
6005    407.673710      92604.079151
2828    410.236847      98022.673693
4580    415.604882     158313.666226

Für Spanien sortiert nach CCreditScore (aufsteigend):
      CCreditScore  CEstimatedSalary
3030      0.000000     187492.510326
199       0.000000      34591.485075
7116    363.927239     102626.457556
8357    409.868237     113755.261608
7402    411.116075      21532.756947
358     411.294329      96630.528457
286     415.464889      60445.952614
6307    417.320459      14701.071458
5957    417.763114     135294.698860
1713    420.403219     144845.021619

Für Frankreich sortiert nach CCreditScore (aufsteigend):
      CCreditScore  CEstimatedSalary
1140      0.000000      29623.040030
6331      0.000000      33964.132028
1539      0.000000      50633.576350
1169      0.000000     183475.501834
1194      0.000000      16581.828641
6965      0.000000     126791.523883
5222    356.805598     147277.646654
8440    406.161731     119471.167072
730     407.937045     113116.196729
6415    408.661830      93918.100254
```

Die Auswertung zeigt eindeutig, dass die Kredit-Scores nicht mit dem Gehalt zusammenhängen können. Deshalb entscheide ich mich dafür, die Nullwerte in den Gehältern mit den mean-values (Mittelwerten) des jeweiligen Landes aufzufüllen. Hierbei entscheide ich mich bewusst für die Gehälter des jeweiligen Landes, da das Lohngefüge unterschiedlich ist.

In [10]:

```python
# Fülle Nullwerte mit den durchschnittlichen Gehältern des jeweiligen Landes

# Null-Werte vor der Behandlung
print("Null-Werte vor der Behandlung:")
for country in ['Germany', 'Spain', 'France']:
    null_count = df[(df['CGeography'] == country) & (df['CEstimatedSalary'].isnu
    print(f"{country}: {null_count}")

# Mit fillna() - die saubere Lösung für alle Länder auf einmal
# observed=True vermeidet FutureWarning bei kategorischen Variablen
df['CEstimatedSalary'] = df.groupby('CGeography', observed=True)['CEstimatedSala
    lambda x: x.fillna(x.mean())
)
```

```
# Null-Werte nach der Behandlung
print("\nNull-Werte nach der Behandlung:")
print(f"CEstimatedSalary gesamt: {df['CEstimatedSalary'].isnull().sum()}")
```

```
Null-Werte vor der Behandlung:
Germany: 162
Spain: 219
France: 397

Null-Werte nach der Behandlung:
CEstimatedSalary gesamt: 0
```

Logische Inkonsistenz: Negatives Gehalt

Ein negatives Gehalt ist wirtschaftlich schlicht nicht möglich. Die betroffenen Zeilen werden deshalb analog zur NaN-Behandlung korrigiert: negativer Wert → auf NaN setzen → länderspezifischer Mittelwert der validen Werte.

In [11]:
```
# Prüfe auf negatives Gehalt (logische Inkonsistenz
negative_salary_count = (df['CEstimatedSalary'] < 0).sum()
print(f"Zeilen mit negativem Gehalt: {negative_salary_count}")

if negative_salary_count > 0:
    print(df[df['CEstimatedSalary'] < 0][['Customer_ID', 'CGeography', 'CEstimat

    # Setze negative Werte auf NaN und imputiere erneut mit dem länderspezifisch
    df.loc[df['CEstimatedSalary'] < 0, 'CEstimatedSalary'] = np.nan
    df['CEstimatedSalary'] = df.groupby('CGeography', observed=True)['CEstimated
        lambda x: x.fillna(x.mean())
    )
    print(f"Nach der Korrektur: Zeilen mit negativem Gehalt: {(df['CEstimatedSal
```

```
Zeilen mit negativem Gehalt: 26
      Customer_ID CGeography  CEstimatedSalary
588   CUST_000589    Germany      -1396.394211
971   CUST_000972     France      -1429.723173
1562  CUST_001563     France      -1911.331355
1605  CUST_001606     France      -1751.124246
1637  CUST_001638    Germany      -1170.710205
2250  CUST_002251     France      -1644.086094
2315  CUST_002316      Spain       -951.134979
3109  CUST_003110     France       -168.021345
3648  CUST_003649     France       -261.638770
4110  CUST_004111      Spain      -1226.987877
4704  CUST_004705     France      -1302.462259
5456  CUST_005457     France      -1484.919392
5813  CUST_005814     France       -107.621118
5872  CUST_005873     France      -3666.109072
6743  CUST_006744      Spain      -1271.130332
6851  CUST_006852     France       -857.616548
6906  CUST_006907    Germany      -1362.378234
7196  CUST_007197     France       -974.864655
7212  CUST_007213    Germany       -797.720776
7499  CUST_007500      Spain       -386.094936
7656  CUST_007657      Spain       -253.465695
7696  CUST_007697     France      -2701.821547
7762  CUST_007763     France      -1618.926727
7856  CUST_007857     France       -106.873614
7903  CUST_007904     France       -709.112943
8256  CUST_008257     France       -203.423292
Nach der Korrektur: Zeilen mit negativem Gehalt: 0
```

In [12]:

```
# Suche einen Zusammenhang zwischen negativem Kontostand und Gehalt, Kredit-Scor
df[df['CBalance'] < 0].sort_values(by=['CBalance'], ascending=True)[['CEstimated
```

Out[12]:

	CEstimatedSalary	CBalance	CCreditScore	CAge
5067	125275.729302	-7218.834421	601.813964	31
6733	101214.306199	-6318.821307	621.521621	34
429	35745.498370	-6232.571927	569.145246	47
6680	89729.235369	-6051.110854	498.370343	47
961	64540.067449	-5770.316433	672.959336	43
...	...	...	...	...
7773	70428.508764	-9.436117	703.551191	44
983	45844.513040	-7.754597	580.184263	38
706	187159.756162	-7.323913	661.633583	20
72	30690.266951	-6.509066	581.795665	37
1558	61854.194911	-0.896123	718.512690	59

1637 rows × 4 columns

Es ist kein Zusammenhang der CBalance erkennbar. Demnach gehe ich davon aus, dass diese irgendwie berechnet wurde. Da kein erklärbares Muster vorliegt, entscheide ich

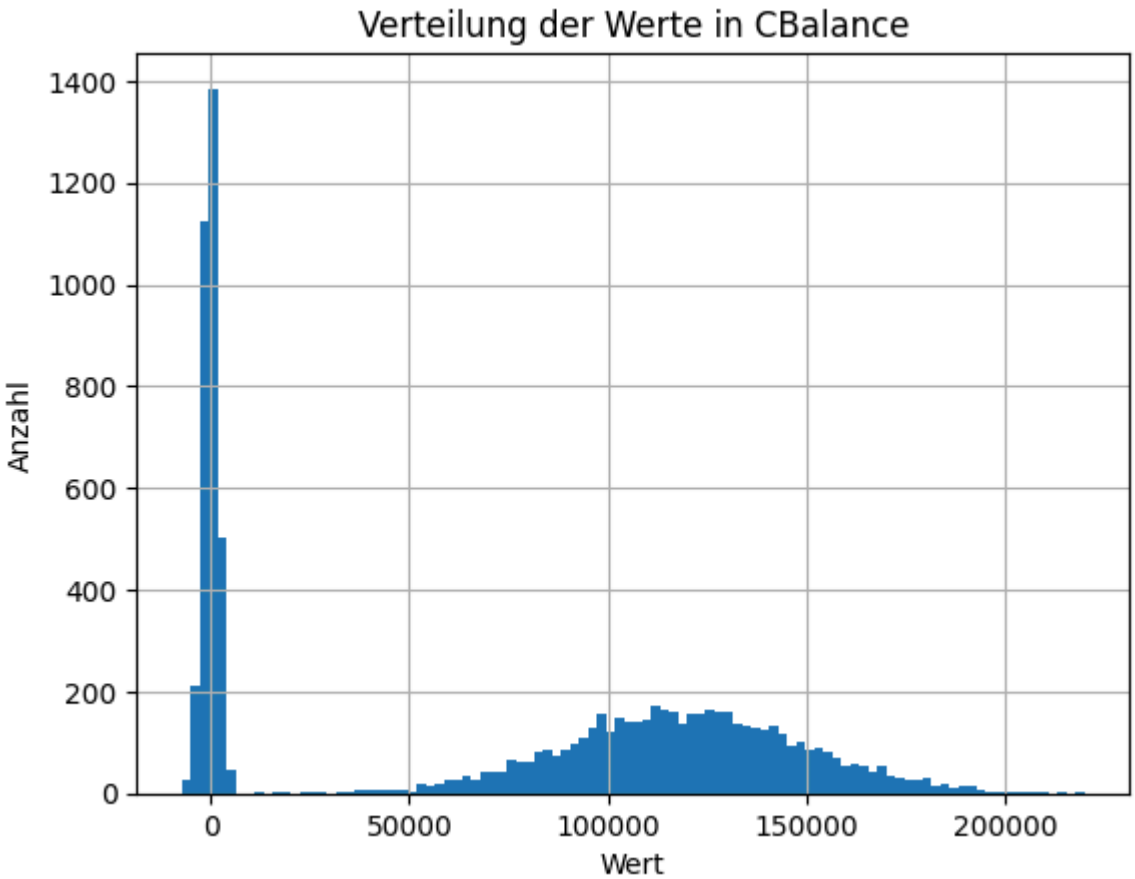
mich für eine **Median-Imputation** der fehlenden Werte. Der Median ist robuster gegenüber Ausreißern als der Mittelwert, was bei der zweigipfligen Verteilung von CBalance (viele Nullen, Rest positiv/negativ) von Vorteil ist.

In [13]:
```
df['CBalance'].isnull().sum()
```

Out[13]:
```
np.int64(77)
```

In [14]:
```
# Überprüfe zunächst auf NaN-Werte
print("Es gibt insgesamt", df['CBalance'].isna().sum(), "NaN Werte in CBalance.\

# Zeige die Verteilung an
plt.hist(df['CBalance'], bins=100)
plt.title("Verteilung der Werte in CBalance")
plt.xlabel("Wert")
plt.ylabel("Anzahl")
plt.grid()
plt.show()

# Median-Imputation für fehlende Werte
median_balance = df['CBalance'].median()
df['CBalance'] = df['CBalance'].fillna(median_balance)

# Dummy-Feature für negative Kontostände
df['CBalance_is_negative'] = np.where(df['CBalance'] < 0, 1, 0)

# Dummy-Feature für Kontostand ≈ 0 (wegen Float-Präzision besser ein Toleranzwer
epsilon = 1e-6
df['CBalance_is_zero'] = np.where(np.abs(df['CBalance']) < epsilon, 1, 0)

# Statistiken ausgeben
print("\nAnzahl der Kunden mit negativem Kontostand: ", df['CBalance_is_negative
print("Anzahl der Kunden mit Kontostand 0: ", df['CBalance_is_zero'].sum())
print("Anzahl der Kunden mit positivem Kontostand: ", df[(df['CBalance_is_negati

# Da Median-Imputation keinen Wert exakt gleich 0 erzeugt, ist CBalance_is_zero
# eine konstante Spalte ohne Informationsgehalt und wird entfernt.
if df['CBalance_is_zero'].nunique() == 1:
    print("\nCBalance_is_zero ist eine konstante Spalte (alle Werte identisch) u
    df.drop('CBalance_is_zero', axis=1, inplace=True)
```

```
Es gibt insgesamt 77 NaN Werte in CBalance.
```




```
Anzahl der Kunden mit negativem Kontostand:  1637
Anzahl der Kunden mit Kontostand 0:  0
Anzahl der Kunden mit positivem Kontostand:  7025

CBalance_is_zero ist eine konstante Spalte (alle Werte identisch) und wird entfer
nt.
```

In [15]:
```
# Überprüfe das Geschlecht auf korrekte Werte
print("Einzigartige Geschlechter vor der Korrektur: ", df['CGender'].unique(), "

# Normalisiere Groß-/Kleinschreibung und führende/nachfolgende Leerzeichen
df['CGender'] = df['CGender'].str.strip().str.capitalize()

# Überprüfe die einzigartigen Geschlechter erneut
print("Einzigartige Geschlechter nach der Korrektur: ", df['CGender'].unique(),

# Konvertiere das Geschlecht in categorischen Datentyp
df['CGender'] = df['CGender'].astype('category')

print("CGender ist jetzt vom Datentyp", df['CGender'].dtype)
```

```
Einzigartige Geschlechter vor der Korrektur:  ['Male' 'Female']

Einzigartige Geschlechter nach der Korrektur:  ['Male' 'Female']

CGender ist jetzt vom Datentyp category
```

In [16]:
```
# Berechne Mittelwert und Standardabweichung des Alters
mean = df['CAge'].mean()
std = df['CAge'].std()

# Definiere die untere und obere Grenze für akzeptable Alterswerte und entferne
lower_bound = mean - 2 * std
```

```
upper_bound = mean + 2 * std

# Berechne minimales und maximales Alter im statistischen 2σ-Bereich
df_filtered = df[(df['CAge'] >= lower_bound) & (df['CAge'] <= upper_bound)]
min_age_stat = df_filtered['CAge'].min()
max_age_stat = df_filtered['CAge'].max()

# Zeige statistische Grenzen transparent an (kann bei fehlerhaften Daten < 18 en
print(f"Statistische Altersgrenzen (2σ): [{lower_bound:.2f}, {upper_bound:.2f}]"
print("Min. Alter im statistischen Bereich:", min_age_stat)
print("Max. Alter im statistischen Bereich:", max_age_stat, "\n")

# Fachliche Altersregel: keine Minderjährigen/negativen Alter
valid_min_age = 18

# Zeige Anzahl der Personen die über/unter den Grenzen liegen (vor Bearbeitung)
print(f"Personen über {max_age_stat} Jahre:", df[df['CAge'] > max_age_stat].shap
print(f"Personen unter {valid_min_age} Jahre:", df[df['CAge'] < valid_min_age].s

# Setze Personen über max_age_stat auf max_age_stat (Capping oberer Rand)
df.loc[df['CAge'] > max_age_stat, 'CAge'] = max_age_stat

# Setze Personen unter 18 auf 18 (Geschäftsfähigkeit/Volljährigkeit)
df.loc[df['CAge'] < valid_min_age, 'CAge'] = valid_min_age

# Überprüfe nach der Bearbeitung
print("Nach der Bearbeitung:")
print(f"Personen über {max_age_stat} Jahre:", df[df['CAge'] > max_age_stat].shap
print(f"Personen unter {valid_min_age} Jahre:", df[df['CAge'] < valid_min_age].s
print("Finale fachliche Altersgrenzen im Datensatz:", df['CAge'].min(), "bis", d
```

```
Statistische Altersgrenzen (2σ): [-8.01, 84.91]
Min. Alter im statistischen Bereich: -1
Max. Alter im statistischen Bereich: 84

Personen über 84 Jahre: 13
Personen unter 18 Jahre: 2

Nach der Bearbeitung:
Personen über 84 Jahre: 0
Personen unter 18 Jahre: 0
Finale fachliche Altersgrenzen im Datensatz: 18 bis 84
```

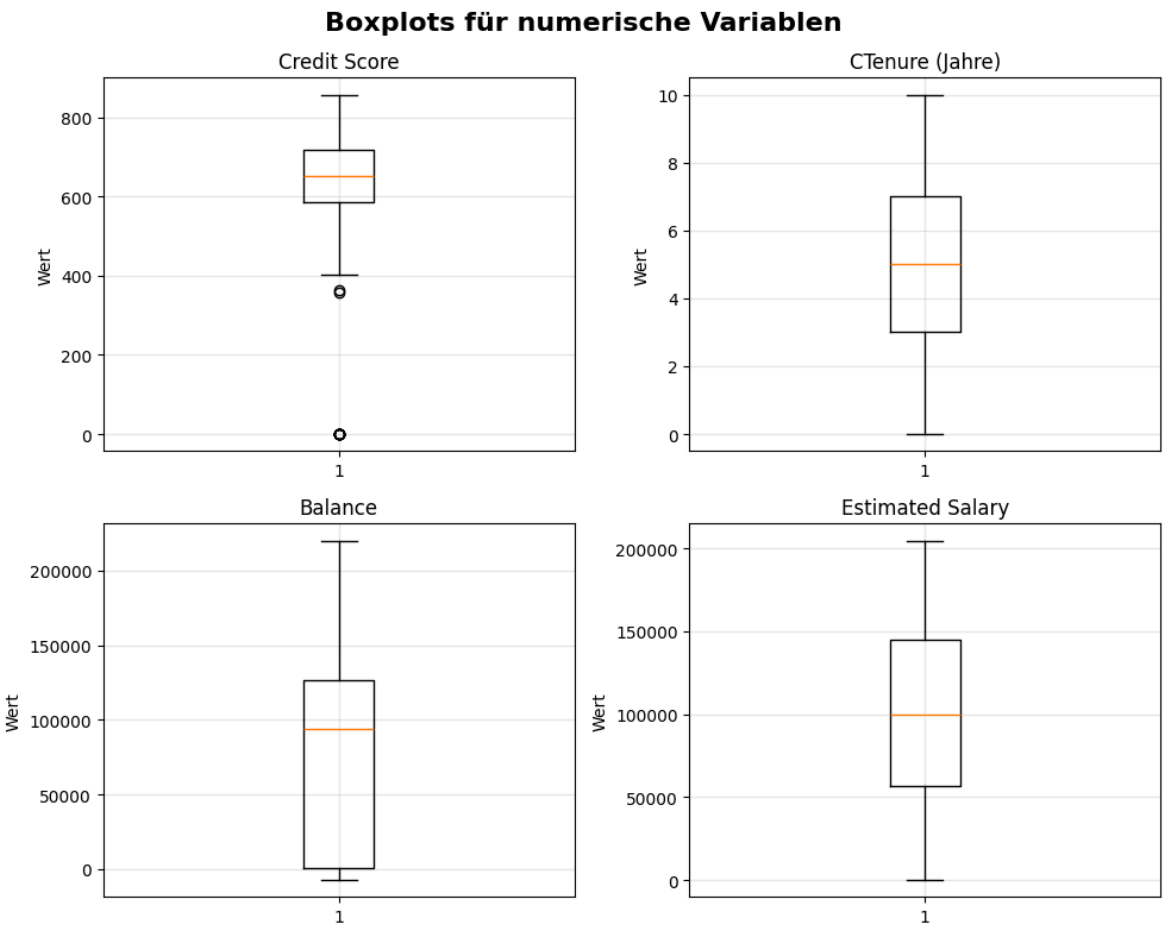
In [17]:

```
# Prüfe Art der Finanzprodukte (Sollte nur 1-4 geben)
print("Vor der Bearbeitung gibt es folgende financial products: ", df['CNumOfPro

# Zeige fehlerhafte Einträge
print("Davon gibt es", df[df['CNumOfProducts'] == 99].count()[['CNumOfProducts']

# Zeige fehlerhafte Einträge
print(df[(df['CNumOfProducts'].isin([18, 99]))])

# Entferne fehlerhafte Einträge mit der Zahl 99 und 18 und prüfe Anzahl Zeilen
df = df[~df['CNumOfProducts'].isin([99, 18])]

# Prüfe Art der Finanzprodukte (Sollte nur 1-4 geben)
print("\n Nach der Bearbeitung gibt es folgende financial products: ", df['CNumO
```

```
Vor der Bearbeitung gibt es folgende financial products:  [1 2 3 4]

Davon gibt es 0 Zeilen mit dem Wert 99.

Empty DataFrame
Columns: [Customer_ID, CCreditScore, CGeography, CGender, CAge, CTenure, CBalanc
e, CNumOfProducts, CHasCrCard, CIsActiveMember, CEstimatedSalary, Days_Since_Onbo
arding, Account_Age_Months, Avg_Monthly_Transactions, Transaction_Variance, Last_
Login_Days_Ago, Support_Tickets_Count, Mobile_App_Usage_Hours, Onboarding_Month,
Is_Holiday_Onboarding, Churn, CLV_Continuous, CBalance_is_negative]
Index: []

[0 rows x 23 columns]

 Nach der Bearbeitung gibt es folgende financial products:  [1 2 3 4]
```

In [18]:

```python
# Boxplots zur Identifikation von Outliers in den demografischen Daten (hint: hä
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
fig.suptitle('Boxplots für numerische Variablen', fontsize=16, fontweight='bold'

# CCreditScore
axes[0, 0].boxplot(df['CCreditScore'])
axes[0, 0].set_title('Credit Score')
axes[0, 0].set_ylabel('Wert')
axes[0, 0].grid(True, alpha=0.3)

# CTenure
axes[0, 1].boxplot(df['CTenure'])
axes[0, 1].set_title('CTenure (Jahre)')
axes[0, 1].set_ylabel('Wert')
axes[0, 1].grid(True, alpha=0.3)

# CBalance
axes[1, 0].boxplot(df['CBalance'])
axes[1, 0].set_title('Balance')
axes[1, 0].set_ylabel('Wert')
axes[1, 0].grid(True, alpha=0.3)

# CEstimatedSalary
axes[1, 1].boxplot(df['CEstimatedSalary'])
axes[1, 1].set_title('Estimated Salary')
axes[1, 1].set_ylabel('Wert')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Zur Behandlung von Ausreißern (Outlier) gibt es verschiedene Methoden. Eine recht einfache Methode, die IQR - Methode soll hier zur Anwendung kommen. In der Praxis werden dabei meist Werte der Inter-Quartil-Range von 1.5 bis 3 angenommen. Aus dem obgigen Boxplot und der Berechnung (siehe unten) wird ersichtlich, dass ein iqr von 1,5 für die Spalte CCreditScore und CTenure ausreichend ist.

In [19]:
```
# Definiere eine Funktion zur Berechnung der Inter Quantile Range (IQR)
def inter_quartile_range(column: pd.Series, iqr: float=1.5) -> tuple:
    """
    Berechnet die Interquartilbereichsgrenzen für eine gegebene Spalte.
    Args:
        column (pd.Series): Die Spalte, für die die IQR-Grenzen berechnet werden
    Returns:
        tuple: Ein Tupel mit dem unteren und oberen Grenzwert des IQR.
    """
    Q3 = np.quantile(column, 0.75)
    Q1 = np.quantile(column, 0.25)
    IQR = Q3 - Q1
    lower_bound = Q1 - iqr * IQR
    upper_bound = Q3 + iqr * IQR
    return lower_bound, upper_bound
```

In [20]:
```
# Berechne Inter Quantile Range (IQR) für CCreditScore
lower_bound, upper_bound = inter_quartile_range(df['CCreditScore'])

# Berechne Anzahl Zeilen vor Bearbeitung
rows_before = df.shape[0]

# Ausgabe der Werte und Grenzen (siehe auch Boxplot oben)
```

```
print("Der untere Wert des Kredit Scores liegt bei: ", df['CCreditScore'].min())
print("Die untere Grenze des Kredit Scores liegt bei: ", lower_bound)
print("Der obere Wert des Kredit Scores liegt bei: ", df['CCreditScore'].max())
print("Die obere Grenze des Kredit Scores liegt bei: ", upper_bound)

# Entferne Ausreißer mit der Inter-Quantile-Range (IQR)-Methode
df = df[(df['CCreditScore'] >= lower_bound) & (df['CCreditScore'] <= upper_bound

# Berechne Anzahl Zeilen nach Bearbeitung
rows_after = df.shape[0]

# Ausgabe
print("Es wurden insgesamt", rows_before - rows_after, "Zeilen entfernt.")
```

```
Der untere Wert des Kredit Scores liegt bei:  0.0
Die untere Grenze des Kredit Scores liegt bei:  384.8683310246385
Der obere Wert des Kredit Scores liegt bei:  857.4577549530931
Die obere Grenze des Kredit Scores liegt bei:  917.031271273406
Es wurden insgesamt 15 Zeilen entfernt.
```

In [21]:

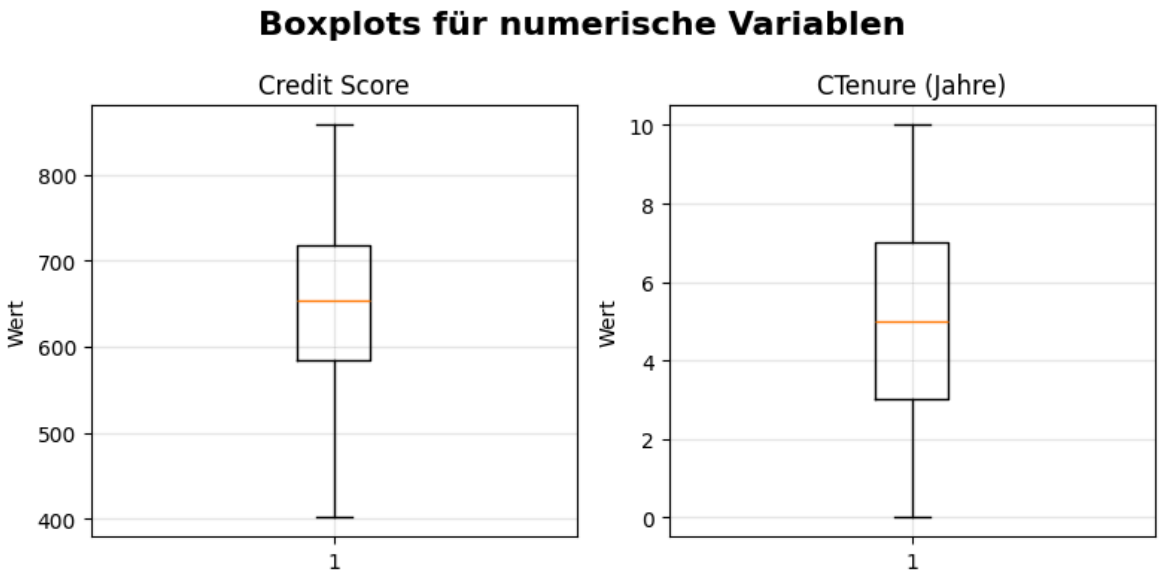
```
# Berechne Inter Quantile Range (IQR) für CTenure
lower_bound, upper_bound = inter_quartile_range(df['CTenure'])

# Berechne Anzahl Zeilen vor Bearbeitung
rows_before = df.shape[0]

# Ausgabe der Werte und Grenzen (siehe auch Boxplot oben)
print("Der untere Wert für die Anzahl der Zugehörigkeit in Jahren liegt bei: ",
print("Die untere Grenze für die Anzahl der Zugehörigkeit in Jahren liegt bei: "
print("Der obere Wert für die Anzahl der Zugehörigkeit in Jahren liegt bei: ", d
print("Die obere Grenze für die Anzahl der Zugehörigkeit in Jahren liegt bei: ",

# Entferne Ausreißer mit der Inter-Quantile-Range (IQR)-Methode
df = df[(df['CTenure'] >= lower_bound) & (df['CTenure'] <= upper_bound)]

# Berechne Anzahl Zeilen nach Bearbeitung
rows_after = df.shape[0]

# Ausgabe
print("Es wurden insgesamt", rows_before - rows_after, "Zeilen entfernt.")
```

```
Der untere Wert für die Anzahl der Zugehörigkeit in Jahren liegt bei:  0
Die untere Grenze für die Anzahl der Zugehörigkeit in Jahren liegt bei:  -3.0
Der obere Wert für die Anzahl der Zugehörigkeit in Jahren liegt bei:  10
Die obere Grenze für die Anzahl der Zugehörigkeit in Jahren liegt bei:  13.0
Es wurden insgesamt 0 Zeilen entfernt.
```

In [22]:

```
# Boxplots zur Überprüfung nach der Entfernung der Outliers
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
fig.suptitle('Boxplots für numerische Variablen', fontsize=16, fontweight='bold'

# CCreditScore
axes[0].boxplot(df['CCreditScore'])
axes[0].set_title('Credit Score')
axes[0].set_ylabel('Wert')
axes[0].grid(True, alpha=0.3)

# CTenure
axes[1].boxplot(df['CTenure'])
axes[1].set_title('CTenure (Jahre)')
axes[1].set_ylabel('Wert')
```

```
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [23]:
```
# Überprüfe die Werte für eine Kreditkarte (erlaubt: 0 oder 1)
print("Werte für CHasCrCard:", df['CHasCrCard'].unique())

# Überprüfe die Werte für eine aktive Mitgliedschaft (erlaubt: 0 oder 1)
print("Werte für CIsActiveMember:", df['CIsActiveMember'].unique())

# Überprüfe Is_Holiday_Onboarding (erwartet: 0 oder 1, keine Null-Werte)
print("\nWerte für Is_Holiday_Onboarding:", df['Is_Holiday_Onboarding'].unique()
print("Null-Werte in Is_Holiday_Onboarding:", df['Is_Holiday_Onboarding'].isnull
print("Verteilung Is_Holiday_Onboarding:")
print(df['Is_Holiday_Onboarding'].value_counts())
```

```
Werte für CHasCrCard: [1 0]
Werte für CIsActiveMember: [0 1]

Werte für Is_Holiday_Onboarding: [0 1]
Null-Werte in Is_Holiday_Onboarding: 0
Verteilung Is_Holiday_Onboarding:
Is_Holiday_Onboarding
0    7148
1    1499
Name: count, dtype: int64
```

Überprüfe und Säubere Temporal & Behavioral Features

In [24]:
```
# Aufgrund von Redundanz und dem Vorschlag für die Spalte Account_Age_Months, we
df.drop(['Days_Since_Onboarding'], axis=1, inplace=True)
print(df.columns)
```

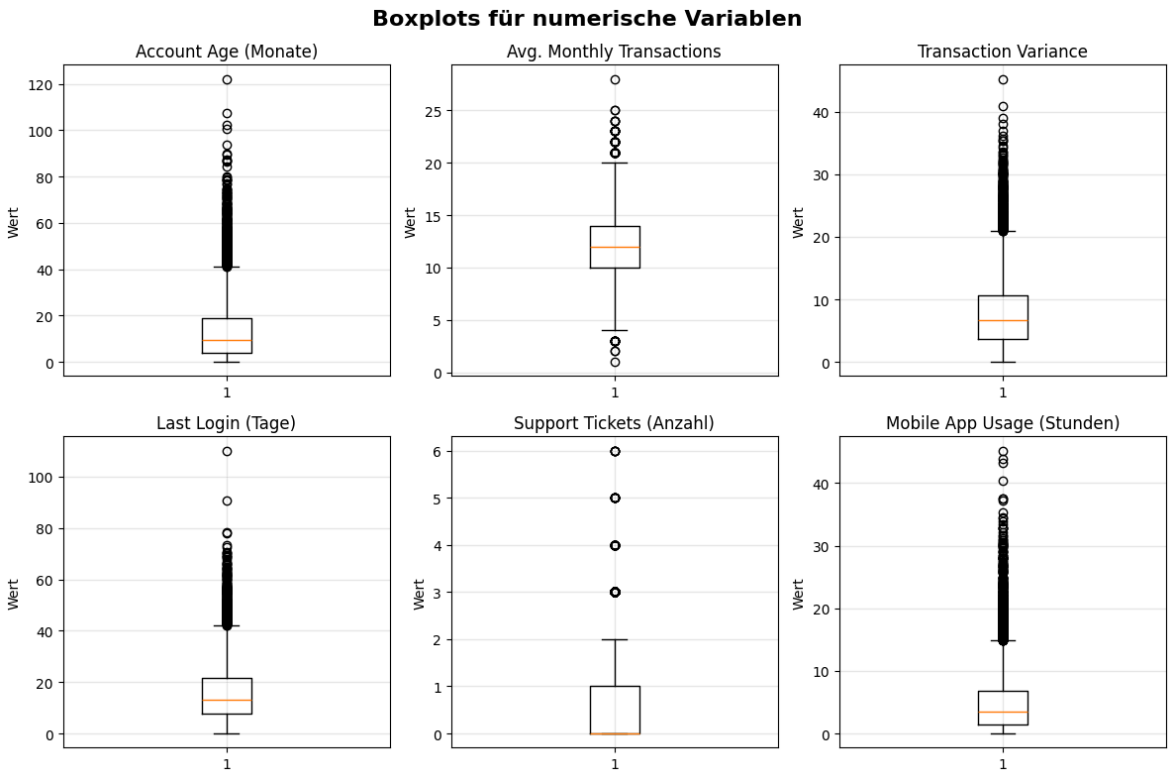
```
Index(['Customer_ID', 'CCreditScore', 'CGeography', 'CGender', 'CAge',
       'CTenure', 'CBalance', 'CNumOfProducts', 'CHasCrCard',
       'CIsActiveMember', 'CEstimatedSalary', 'Account_Age_Months',
       'Avg_Monthly_Transactions', 'Transaction_Variance',
       'Last_Login_Days_Ago', 'Support_Tickets_Count',
       'Mobile_App_Usage_Hours', 'Onboarding_Month', 'Is_Holiday_Onboarding',
       'Churn', 'CLV_Continuous', 'CBalance_is_negative'],
      dtype='object')
```

In [25]:

```python
# Überprüfe die Werte in der Spalte Onboarding_Month
print("Es gibt folgende Werte: ", df['Onboarding_Month'].unique(), "\n")

# Pattern-Matching
num_months = {1: "January", 2: "February", 3: "March", 4: "April", 5: "May", 6:
            8: "August", 9: "September", 10: "October", 11: "November", 12: "Dec

seasons = {
    'Winter': ['December', 'January', 'February'],
    'Spring': ['March', 'April', 'May'],
    'Summer': ['June', 'July', 'August'],
    'Autumn': ['September', 'October', 'November']
}

# Mapping-Funktion für die Jahreszeiten
def get_season(month: int) -> str:
    month_name = num_months[month]
    for season, months in seasons.items():
        if month_name in months:
            return season
    return "Unknown"

# Neue Spalte CSeason mit Jahreszeiten befüllen und zum Datensatz hinzufügen
df['CSeason'] = df['Onboarding_Month'].apply(get_season).astype('category')

# Ausgabe (Summary)
print("Unique seasons assigned: ", df['CSeason'].unique(), "\n")
print("Verteilung der Jahreszeiten:")
print(df['CSeason'].value_counts())
print(f"\nEs wurden insgesamt {df.shape[0]} Zeilen mit Jahreszeiten befüllt.\n")

# Wandle die Jahreszeiten in categorical
df['CSeason'] = df['CSeason'].astype('category')
print("Der Datentyp von CSeason ist nun", df['CSeason'].dtype)
```

```
Es gibt folgende Werte:  [ 8 11  2  1  3  5  7 10  9  4  6 12]

Unique seasons assigned:  ['Summer', 'Autumn', 'Winter', 'Spring']
Categories (4, object): ['Autumn', 'Spring', 'Summer', 'Winter']

Verteilung der Jahreszeiten:
CSeason
Winter    2194
Summer    2187
Spring    2151
Autumn    2115
Name: count, dtype: int64

Es wurden insgesamt 8647 Zeilen mit Jahreszeiten befüllt.

Der Datentyp von CSeason ist nun category
```

In [26]:
```
# Boxplots zur Identifikation von Ausreißern bei den Temporal & Behavioral Featu
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
fig.suptitle('Boxplots für numerische Variablen', fontsize=16, fontweight='bold'

# CAccount_Age_Months
axes[0, 0].boxplot(df['Account_Age_Months'])
axes[0, 0].set_title('Account Age (Monate)')
axes[0, 0].set_ylabel('Wert')
axes[0, 0].grid(True, alpha=0.3)

# CAvg_Monthly_Transactions
axes[0, 1].boxplot(df['Avg_Monthly_Transactions'])
axes[0, 1].set_title('Avg. Monthly Transactions')
axes[0, 1].set_ylabel('Wert')
axes[0, 1].grid(True, alpha=0.3)

# CTransaction_Variance
axes[0, 2].boxplot(df['Transaction_Variance'])
axes[0, 2].set_title('Transaction Variance')
axes[0, 2].set_ylabel('Wert')
axes[0, 2].grid(True, alpha=0.3)

# Last Login
axes[1, 0].boxplot(df['Last_Login_Days_Ago'])
axes[1, 0].set_title('Last Login (Tage)')
axes[1, 0].set_ylabel('Wert')
axes[1, 0].grid(True, alpha=0.3)

# Support Tickets
axes[1, 1].boxplot(df['Support_Tickets_Count'])
axes[1, 1].set_title('Support Tickets (Anzahl)')
axes[1, 1].set_ylabel('Wert')
axes[1, 1].grid(True, alpha=0.3)

# Mobile Usage
axes[1, 2].boxplot(df['Mobile_App_Usage_Hours'])
axes[1, 2].set_title('Mobile App Usage (Stunden)')
axes[1, 2].set_ylabel('Wert')
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Insgesamt beginnen Werte minimum bei 0. Die Spalte "Account_Age_Months" wird nicht mit IQR behandelt, da es sich bei Ausreißern um Langzeitkunden handeln könnte. Demnach würden wichtige Informationen entfernt. Genauso würden Kunden, die lange nicht eingeloggt waren, aber nicht gekündigt haben, ein besonderes Nutzungsmuster darstellen. Ggf. sind dies "passive" Kunden, die das Konto behalten, aber kaum nutzen. Für die Anzahl der Tickets würde ich eine IQR von 3 vorschlagen (nicht so extrem). Mehr Tickets könnten darauf hindeuten, dass es zu Fehlern beim erstellen gekommen ist, wie zum Beispiel durch Duplikate oder es wurden Tickets abgesendet, aber nicht richtig ausgefüllt (Neuerstellung notwendig für die gleiche Transaktion). Bei den App-Nutzern würde ich die Werte belassen. Es gibt eben Personen die sehr oft auf ihr Konto schauen und Personen die dies weniger tun. Insgesamt gibt es keine rießigen Ausreißer. Deshalb erscheinen die Daten valide. Die monatlichen Transaktionen könnten mit der IQR - Methode bereinigt werden. Dadurch würde die Varianz der Transaktionen weniger streuen.

In [27]:
```
# Zähle die Anzahl der Zeilen vor der Bearbeitung
rows_before = df.shape[0]

# Berechne Inter Quantile Range (IQR) für Support_Tickets_Count
lower_bound, upper_bound = inter_quartile_range(column=df['Support_Tickets_Count
print("Support_Tickets_Count - Untere Grenze:", lower_bound)
print("Support_Tickets_Count - Obere Grenze:", upper_bound)

# Entferne Ausreißer mit der Inter-Quantile-Range (IQR)-Methode
df = df[(df['Support_Tickets_Count'] >= lower_bound) & (df['Support_Tickets_Coun

# Zähle die Anzahl der Zeilen nach der Bearbeitung
rows_after = df.shape[0]

# Ausgabe
print("Es wurden insgesamt", rows_before - rows_after, "Zeilen entfernt.")
```

```
# change into int value
df['Support_Tickets_Count'] = df['Support_Tickets_Count'].astype('int')
```

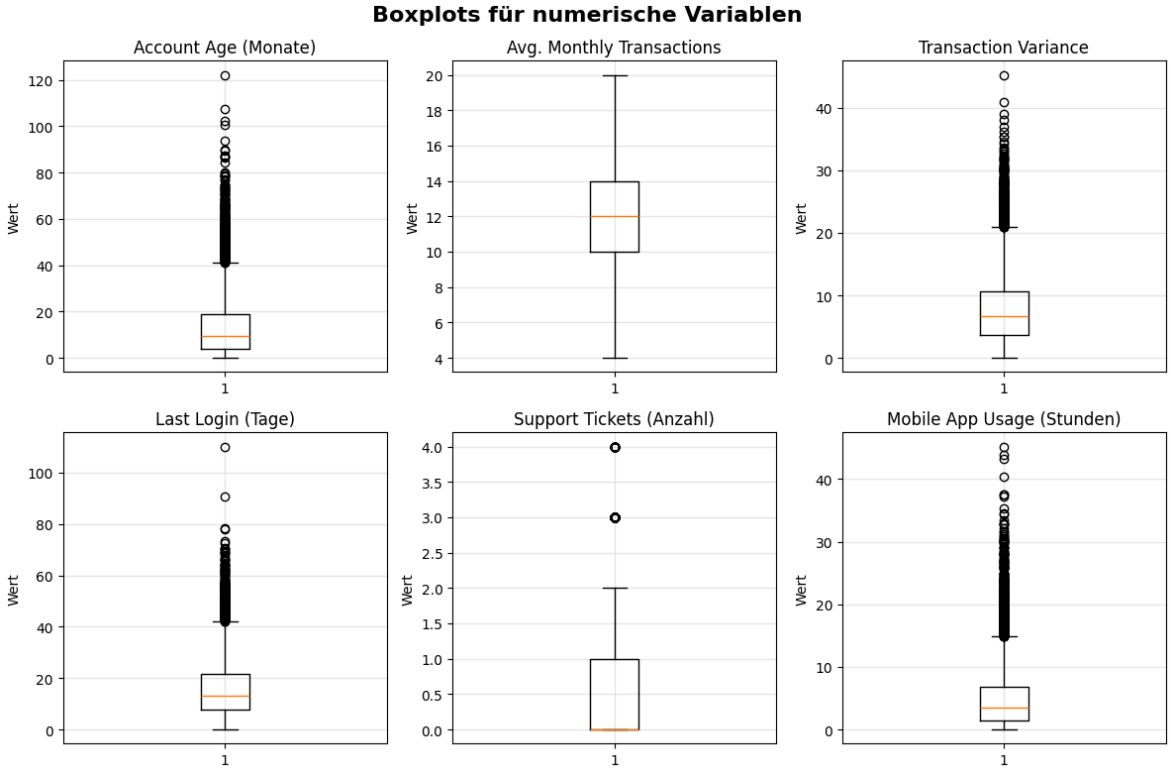
```
Support_Tickets_Count - Untere Grenze: -3.0
Support_Tickets_Count - Obere Grenze: 4.0
Es wurden insgesamt 18 Zeilen entfernt.
```

In [28]:
```
# Zähle die Anzahl der Zeilen vor der Bearbeitung
rows_before = df.shape[0]

# Berechne Inter Quantile Range (IQR) für Avg_Monthly_Transactions
lower_bound, upper_bound = inter_quartile_range(column=df['Avg_Monthly_Transacti
print("Avg_Monthly_Transactions - Untere Grenze:", lower_bound)
print("Avg_Monthly_Transactions - Obere Grenze:", upper_bound)

# Entferne Ausreißer mit der Inter-Quantile-Range (IQR)-Methode
df = df[(df['Avg_Monthly_Transactions'] >= lower_bound) & (df['Avg_Monthly_Trans

# Zähle die Anzahl der Zeilen nach der Bearbeitung
rows_after = df.shape[0]

# Ausgabe
print("Es wurden insgesamt", rows_before - rows_after, "Zeilen entfernt.")
```

```
Avg_Monthly_Transactions - Untere Grenze: 4.0
Avg_Monthly_Transactions - Obere Grenze: 20.0
Es wurden insgesamt 108 Zeilen entfernt.
```

In [29]:
```
# Boxplots zur Identifikation von Ausreißern bei den Temporal & Behavioral Featu
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
fig.suptitle('Boxplots für numerische Variablen', fontsize=16, fontweight='bold'

# CAccount_Age_Months
axes[0, 0].boxplot(df['Account_Age_Months'])
axes[0, 0].set_title('Account Age (Monate)')
axes[0, 0].set_ylabel('Wert')
axes[0, 0].grid(True, alpha=0.3)

# CAvg_Monthly_Transactions
axes[0, 1].boxplot(df['Avg_Monthly_Transactions'])
axes[0, 1].set_title('Avg. Monthly Transactions')
axes[0, 1].set_ylabel('Wert')
axes[0, 1].grid(True, alpha=0.3)

# CTransaction_Variance
axes[0, 2].boxplot(df['Transaction_Variance'])
axes[0, 2].set_title('Transaction Variance')
axes[0, 2].set_ylabel('Wert')
axes[0, 2].grid(True, alpha=0.3)

# Last Login
axes[1, 0].boxplot(df['Last_Login_Days_Ago'])
axes[1, 0].set_title('Last Login (Tage)')
axes[1, 0].set_ylabel('Wert')
axes[1, 0].grid(True, alpha=0.3)

# Support Tickets
axes[1, 1].boxplot(df['Support_Tickets_Count'])
axes[1, 1].set_title('Support Tickets (Anzahl)')
axes[1, 1].set_ylabel('Wert')
```

```
axes[1, 1].grid(True, alpha=0.3)

# Mobile Usage
axes[1, 2].boxplot(df['Mobile_App_Usage_Hours'])
axes[1, 2].set_title('Mobile App Usage (Stunden)')
axes[1, 2].set_ylabel('Wert')
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Mit IQR von 1.5 wurden insgesamt zwar 108 Zeilen entfernt, aber die monatliche Transaktionsrate besteht nun ohne Ausreißer. Insgesamt können Transaktionen zwar einen niedrigen Durchschnitt (z.B. 5 Transaktionen/Monat), aber eine hohe Varianz haben (mal 0, mal 15 Transaktionen). Dabei handelt es sich um ein legitimes Verhaltensmuster und keinen Fehler. Beispielsweise würde ein Student selten Transaktionen ausführen (niedriger Avg). Sobald er Geld bekommt, würde er viele Transaktionen durchführen (hohe Varianz). Weitere Beispiele erklären sich von selbst. Demnach ist die Variance mehr oder weniger vairabel/flexibel und besitzt eine höhere Anzahl von "Ausreißern". Aus diesem Grund würde ich die Varianz auch nicht weiter behandeln.

Bei einigen Temporal & Behavioral Features wurde bewusst auf eine Ausreißer-Entfernung verzichtet, da hohe Werte in Account_Age_Months Langzeitkunden darstellen (wichtig für Retention-Analyse). Extreme Werte in Last_Login_Days_Ago repräsentieren das wichtigste Churn-Signal. Hohe Mobile_App_Usage_Hours deuten auf engagierte, loyale Kunden hin. Lediglich Support_Tickets_Count wurde mit IQR 3.0 behandelt, um offensichtliche Systemfehler (z.B. Duplikate) zu entfernen.

In [30]:
```
# Kompakter Skewness-Check
skew_cols = [
    'CAge', 'Account_Age_Months', 'Transaction_Variance',
    'Last_Login_Days_Ago', 'Support_Tickets_Count', 'Mobile_App_Usage_Hours'
```

```
]

skew_table = df[skew_cols].skew().sort_values(ascending=False).round(3)
print("Skewness (ausgewählte Features):")
print(skew_table)

high_skew = skew_table[skew_table.abs() >= 1]
print("\nStark schief (|skew| >= 1):", list(high_skew.index))
print("Entscheidung: Transformation nur modellabhängig prüfen (v. a. lineare/dis
```

```
Skewness (ausgewählte Features):
Mobile_App_Usage_Hours    2.098
Account_Age_Months        1.842
Support_Tickets_Count     1.610
Last_Login_Days_Ago       1.418
Transaction_Variance      1.364
CAge                      1.257
dtype: float64

Stark schief (|skew| >= 1): ['Mobile_App_Usage_Hours', 'Account_Age_Months', 'Sup
port_Tickets_Count', 'Last_Login_Days_Ago', 'Transaction_Variance', 'CAge']
Entscheidung: Transformation nur modellabhängig prüfen (v. a. lineare/distanzbasi
erte Modelle), nicht global.
```

Skewness-Check und Transformationsentscheidung

Sechs Features weisen eine Schiefe von |skew| ≥ 1 auf (u. a. `Mobile_App_Usage_Hours` : 2.10, `Account_Age_Months` : 1.84). Für lineare und distanzbasierte Modelle (z. B. kNN, Logistische Regression) wäre eine `log1p` -Transformation grundsätzlich sinnvoll, da sie die Verteilung symmetrischer macht und Distanzberechnungen weniger verzerrt.

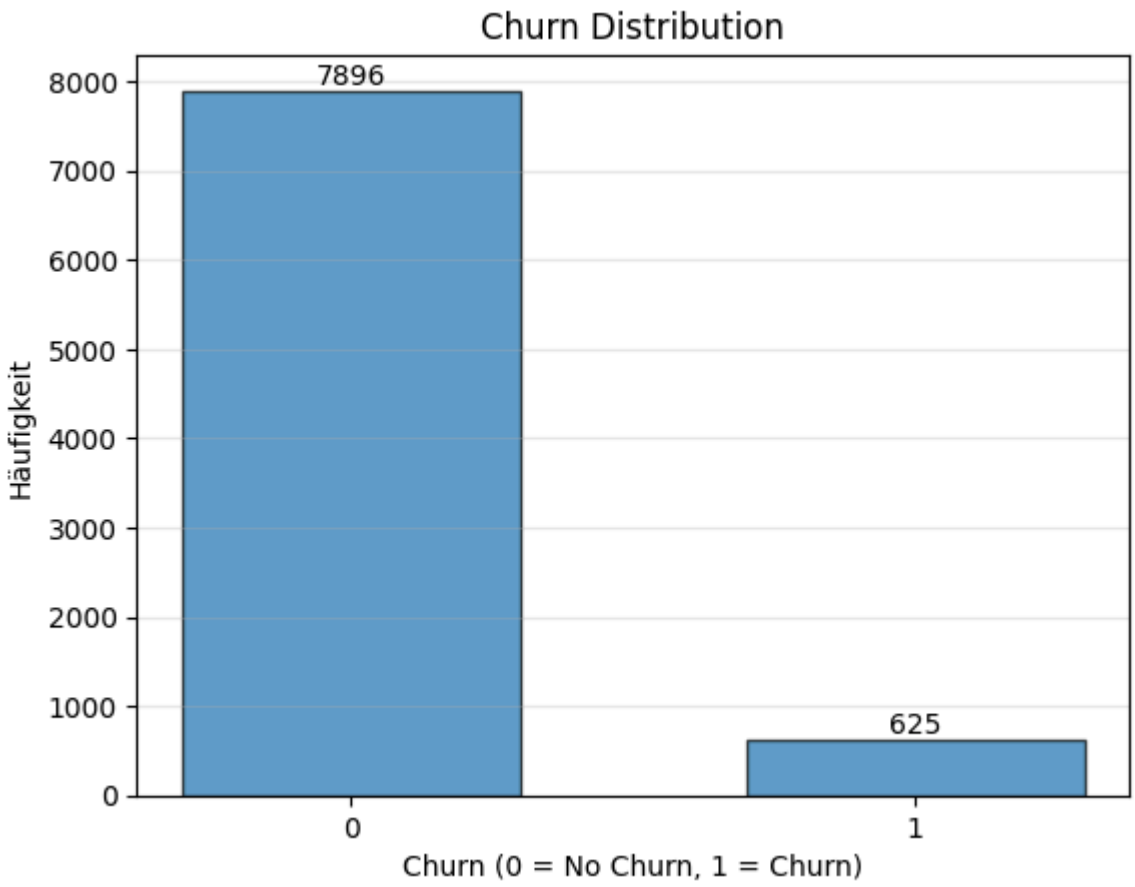
Da in dieser Arbeit jedoch primär **baumbasierte Modelle** (Decision Tree, Random Forest) verwendet werden, die gegenüber monotonen Transformationen invariant sind, wird darauf verzichtet. Für kNN und Logistische Regression wird stattdessen eine Skalierung via `StandardScaler` angewendet, die den größten Teil der Verzerrung durch unterschiedliche Wertebereiche kompensiert.

Überprüfe und Säubere Target Variables

In [31]:
```
# Überprüfe die Spalte Churn
print("Es gibt für die Spalte Churn insgesamt folgende Werte:", df['Churn'].uniq

# Füge eine neue (extra) Spalte Churn_Category zum Datensatz hinzu
df['Churn_Category'] = df['Churn'].apply(lambda x: 'No Churn' if x == 0 else 'Ch

# Prüfe die neue Spalte Churn_Category
print("Es wurden insgesamt folgende Churn-Kategorien gefunden:", df['Churn_Categ
print("Es wurden insgesamt:", len(df['Churn_Category']), "Zeilen hinzugefügt. Vo

# BarChart der Spalte Churn mit den Klassen 0 und 1
churn_counts = df['Churn'].value_counts().sort_index()
bars = plt.bar(churn_counts.index, churn_counts.values, edgecolor='black', alpha
plt.title('Churn Distribution')
plt.xlabel('Churn (0 = No Churn, 1 = Churn)')
plt.ylabel('Häufigkeit')
plt.xticks([0, 1])
```

```
plt.grid(axis='y', alpha=0.3)
bars = plt.bar_label(bars, fmt='%d')
plt.show()
```

```
Es gibt für die Spalte Churn insgesamt folgende Werte: [0 1]

Es wurden insgesamt folgende Churn-Kategorien gefunden: ['No Churn', 'Churn']
Categories (2, object): ['Churn', 'No Churn']

Es wurden insgesamt: 8521 Zeilen hinzugefügt. Von insgesamt: 8521
```



Das sog. "target label" für Churn ist unbalanciert. Nur rund 7,9% der Datensätze sind Churn = 1. D. h. wenig Kunden wandern ab (positiv). Im Gegensatz dazu sind No Churn labels mit 92,1% im Datensatz vertreten. Das bedeutet die Machine Learning Modelle werden Churns (Minderheitsklasse) nicht gut bzw. richtig prädizieren. Folglich würde die Accuracy zwar hoch ausfallen, aber nur, weil die Mehrheitsklasse richtig prädiziert würde. Anders ausgedrückt würde überwiegend die Mehrheitsklasse richtig prädiziert werden. Allgemein sollte eine Accuracy von z.B. über 90% jeden Anwender eher stutzig machen.

Eine Methode damit umzugehen ist SMOTE (Synthetic minority oversampling technique). Hierbei werden synthetische Daten erzeugt, ohne ein overfitting, also eine Überanpassung, des Modells zu erzeugen. Im Gegensatz dazu führt ein Resampling (Random-Oversamplig) eher zur Überanpassung des Modells. Eine weitere Methode wäre das sog. Undersampling, bei dem Daten aus der Mehrheitsklasse entfernt würden, bis eine gleichmäßige Durchmischung erreicht ist. Dadurch würde die Gesamtzahl der Samples minimiert und der Datensatz insgesamt kleiner werden. Schließlich würde ich mich für SMOTE entscheiden, wenn ich zu task 2 gelange und feststelle, dass die Performance der Modelle nicht gut ist.

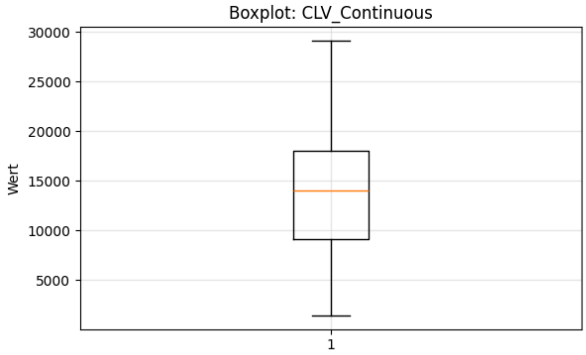
Hinweis: Random Forests und die Logistische Regression werden als Modelle deklariert, die gut mit kleinen und unbalancierten Datensätzen umgehen können.

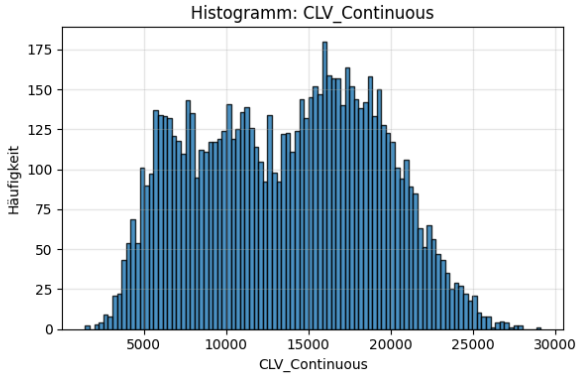
source1

source2

In [32]:
```
# Überprüfe die Spalte CLV_Continuous
print("=== Statistik === ")
print(df['CLV_Continuous'].describe(), "\n")
print("Anzahl Null Werte:", df['CLV_Continuous'].isna().sum())
print("Anzahl negative Werte:", df['CLV_Continuous'].apply(lambda x: x < 0).sum(
print("Anzahl positive Werte:", df['CLV_Continuous'].apply(lambda x: x > 0).sum(
print("Schiefe", round(skew(df['CLV_Continuous']), 3))
print("Kurtosis", round(kurtosis(df['CLV_Continuous']), 3))

# Boxplot und Histogramm nebeneinander
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].boxplot(df['CLV_Continuous'])
axes[0].set_title('Boxplot: CLV_Continuous')
axes[0].set_ylabel('Wert')
axes[0].grid(True, alpha=0.3)

axes[1].hist(df['CLV_Continuous'], bins=100, edgecolor='black', alpha=0.8)
axes[1].set_title('Histogramm: CLV_Continuous')
axes[1].set_xlabel('CLV_Continuous')
axes[1].set_ylabel('Häufigkeit')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
=== Statistik ===
count     8521.000000
mean     13674.753402
std       5448.529740
min       1428.027822
25%       9066.356904
50%      14030.258672
75%      18008.789676
max      29123.502825
Name: CLV_Continuous, dtype: float64

Anzahl Null Werte: 0
Anzahl negative Werte: 0
Anzahl positive Werte: 8521
Schiefe -0.001
Kurtosis -0.976
```





`CLV_Continuous` ist unauffällig: Schiefe nahe 0, Kurtosis von −0.976, keine Nullwerte, keine negativen Werte, keine Ausreißer im Boxplot. Das ist das sauberste Feature im gesamten Datensatz und braucht keine weitere Behandlung.

In [33]:
```
# Zeige die Anzahl der Zeilen, die durch das Data-Cleaning beseitigt werden muss
rows_at_end = df.shape[0]

print("Es wurden insgesamt:", rows_at_begin - rows_at_end, "Zeilen durch das Dat
print("Der Datensatz enthält nun insgesamt:", df.shape[0], "Zeilen und", df.shap
```

```
Es wurden insgesamt: 141 Zeilen durch das Data-Cleaning entfernt.
Der Datensatz enthält nun insgesamt: 8521 Zeilen und 24 Spalten.
```

Feature Engineering: Dummie-Encoding für kategorische Variablen

In [34]:
```
# Setze Dummie-Encoding ein: Werte (0 | 1) für kategorische Variablen ein, wie C
# drop_first=True verhindert Multikollinearität, da die erste Spalte weggelassen
df = pd.get_dummies(df, columns=['CGeography', 'CGender', 'CSeason'], drop_first

# WICHTIG: Entferne redundante und problematische Spalten für ML-Datensatz
# - Onboarding_Month: Redundant (CSeason vorhanden)
# - Churn_Category: Redundant (Kopie von Churn)
# - Customer_ID: Identifier, nicht für ML relevant
df.drop(['Onboarding_Month', 'Churn_Category', 'Customer_ID'], axis=1, inplace=T
```

Runde auf volle Zahlen (ohne Komma) für: Gehälter, avg. Monate des Accounts, Last_Login_Days_Ago

In [35]:
```
# Es genügt die Spalte der Gehälter und auch Monate auf volle Integer zu runden
df['CEstimatedSalary'] = round(df['CEstimatedSalary']).astype('int')
df['Account_Age_Months'] = round(df['Account_Age_Months']).astype('int')
df['Last_Login_Days_Ago'] = round(df['Last_Login_Days_Ago']).astype('int')
```

Runde auf weniger Nachkommastellen für den Credit-Score und Mobile_App_Usage_Hours

In [36]:
```
df['CCreditScore'] = round(df['CCreditScore'], 3).astype('float')
df['Mobile_App_Usage_Hours'] = round(df['Mobile_App_Usage_Hours'], 2).astype('fl
# df.head()
```

Schreibe die gesäuberten Daten in eine CSV-Datei `dataset_cleaned.csv` im Ordner `data`

In [37]:
```
# Schreibe den ML-Modell gesäuberten Datensatz als csv zurück
df.to_csv("../data/dataset_cleaned.csv", index=False)
```

An dieser Stelle ist das sog. Data-Cleaning abgeschlossen!